

Apache Iceberg Parquet

模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块: Parquet (iceberg-parquet)

版本: apache-iceberg-1.9.2

生成工具: AI Architecture Analyzer

生成日期: 2026年03月24日

第1章 模块概述

1.1 模块定位

Apache Iceberg 是一种高性能的开放表格式（Open Table Format），专为大规模分析型数据集设计。iceberg-parquet 模块是 Iceberg 的核心存储格式支持模块之一，负责将 Iceberg 的逻辑表模型与 Apache Parquet 列式存储文件格式进行桥接。它实现了 Parquet 文件的完整读写生命周期管理，包括 Schema 转换、数据读写、行组过滤、指标收集、向量化读取、Variant 半结构化类型支持以及加密等功能。

1.2 核心能力

- Schema 双向转换 - Iceberg Schema 与 Parquet MessageType 之间的双向无损转换
- 高性能数据读取 - 支持行式和向量化批量两种读取模式，配合三级行组过滤策略
- 灵活数据写入 - 支持数据写入、删除文件写入，可配置压缩、布隆过滤器、加密等
- 丰富类型支持 - 覆盖所有 Iceberg 数据类型，含 Variant 半结构化数据的分片读写
- IO 适配层 - 将 Iceberg 统一 IO 接口适配到 Parquet 底层 IO，支持向量化 IO
- 指标收集 - 从 Parquet footer 和写入过程收集列级统计信息用于查询优化

1.3 源码文件一览

该模块共包含 58 个主源码文件和 18 个测试文件，分布在 5 个包中。以下列出核心源码文件：

文件名	行数	职责描述
Parquet.java	~1640	模块主 API 入口，提供 read/write/writeData/writeDeletes 静态工厂及 Builder
ParquetReader.java	~177	行式 Parquet 读取器，CloseableIterable 按行组迭代
ParquetWriter.java	~268	Parquet 写入器，FileAppender 管理行组刷新
ParquetValueReaders.java	~1221	值读取器体系: Primitive/Struct/List/Map/OptionReader 等
ParquetValueWriters.java	~732	值写入器体系: Primitive/Struct/Collection/MapWriter 等
ParquetSchemaUtil.java	~284	Schema 转换工具: convert/pruneColumns/addFallbackIds
ParquetTypeVisitor.java	~274	Parquet Schema 通用访问者
TypeWithSchemaVisitor.java	~263	双 Schema 同步访问者
MessageTypeToType.java	~264	Parquet MessageType 转 Iceberg Type
TypeToMessageType.java	~291	Iceberg Schema 转 Parquet MessageType

文件名	行数	职责描述
ParquetMetricsRowGroupFilter.java	~599	统计信息行组过滤器 (min/max)
ParquetDictionaryRowGroupFilter.java	~480	字典编码行组过滤器
ParquetBloomRowGroupFilter.java	~352	布隆过滤器行组过滤器
ParquetFilters.java	~253	Iceberg 表达式转 Parquet FilterPredicate
ParquetIO.java	~266	IO 适配层: Iceberg IO 到 Parquet IO
ParquetMetrics.java	~647	指标收集: 从 footer 提取列级统计
ReadConf.java	~223	读取配置: 三级过滤 + Schema 投影
VectorizedParquetReader.java	~177	向量化批量读取器
BaseParquetReaders.java	~415	抽象基类: 模板方法构建读取器
BaseParquetWriter.java	~274	抽象基类: 模板方法构建写入器
GenericParquetReaders.java	~257	具体实现: Record 读取器
GenericParquetWriter.java	~171	具体实现: Record 写入器

第2章 核心架构设计

2.1 整体分层架构

Iceberg Parquet 模块采用清晰的四层架构设计: API
层（对外统一接口）、核心层（读写器/Schema/过滤器）、实现层（类型特定的读写器实现）、IO
层（文件系统适配）。层间通过接口和抽象类解耦，依赖方向严格自上而下。

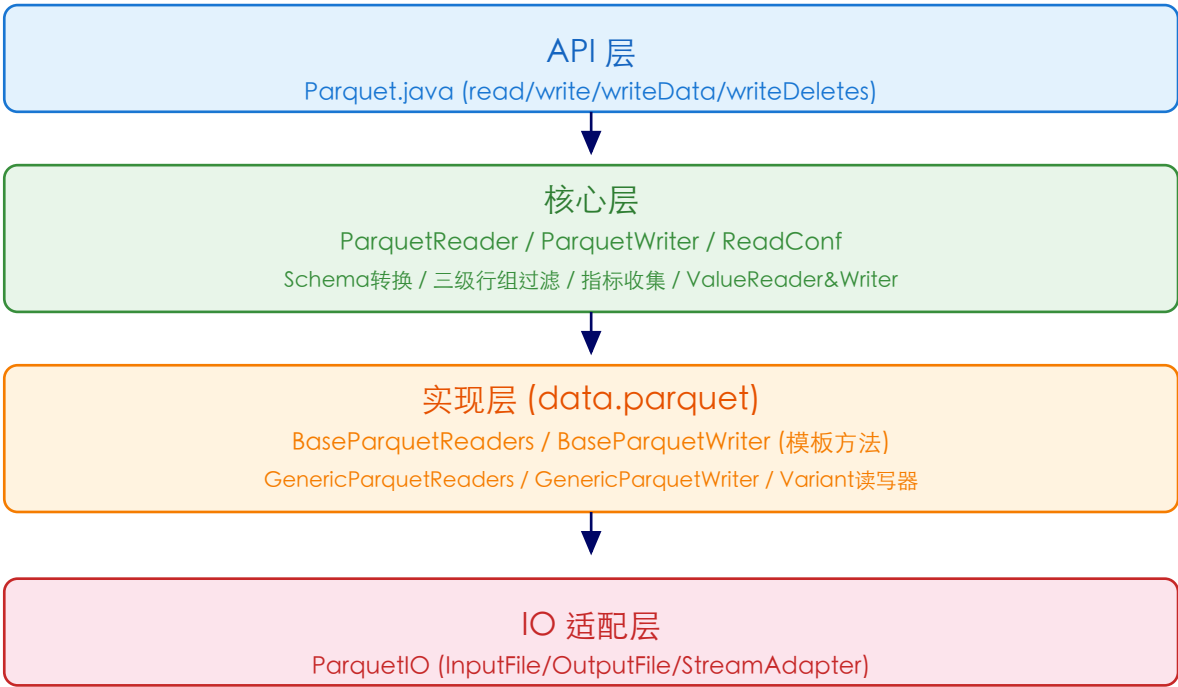


图 2-1 Iceberg Parquet 模块四层架构图

2.2 核心组件关系

Parquet.java 作为门面提供所有对外 API; Builder 使用建造者模式组装配置; ParquetReader 使用 ReadConf 加载配置并应用三级过滤; Schema 转换通过 TypeVisitor 访问者模式实现; 值读写器通过组合模式处理嵌套类型。

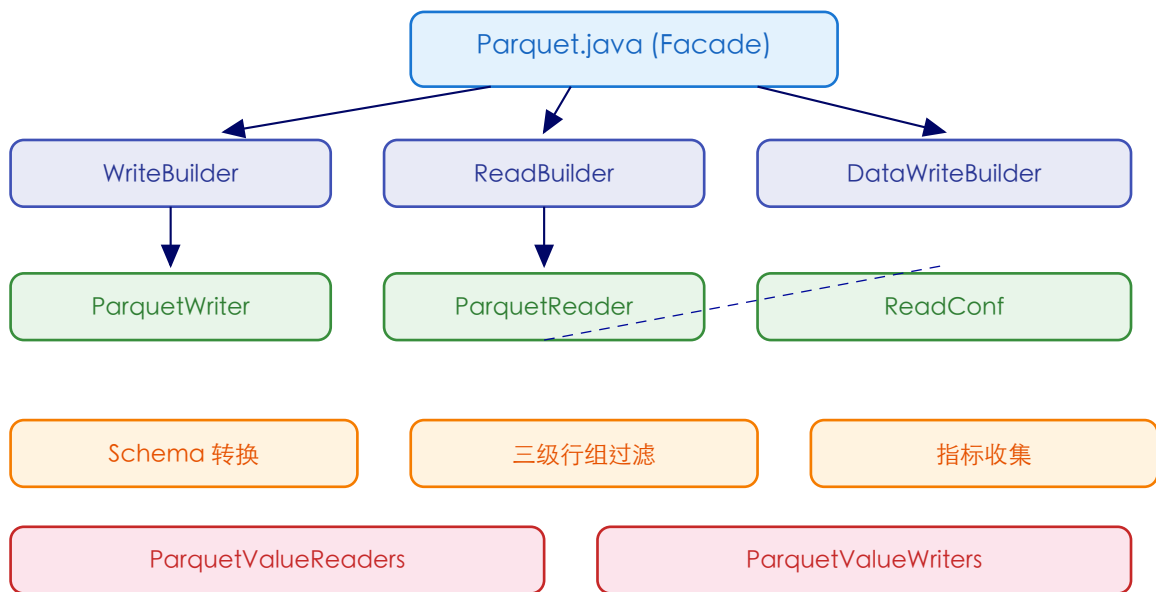


图 2-2 核心组件协作关系图

2.3 包结构

包名	文件数	核心职责
o.a.iceberg.parquet	29	主包: 读写器、Schema转换、过滤器、IO适配、指标收集
o.a.iceberg.parquet.data.parquet	12	数据层: 抽象基类和具体读写器实现
o.a.iceberg.parquet (Variant)	6	Variant 半结构化数据支持
o.a.iceberg.data.parquet (internal)	5	内部数据框架集成
o.a.iceberg.parquet (Avro)	6	Avro 兼容层和逻辑类型转换

第3章 核心类设计模式详解

3.1 Parquet.java - 建造者模式 (Builder) + 门面模式 (Facade)

Parquet.java 是模块的 API 门面，通过静态工厂方法 write()、read()、writeData()、writeDeletes() 创建 Builder 内部类。每个 Builder 采用流式 API 设计，链式配置压缩、行组大小、布隆过滤器等参数，最终 build() 构建实例。

- WriteBuilder - 通用写入构建器: Schema、压缩、行组大小、布隆过滤器等
- ReadBuilder - 通用读取构建器: 投影 Schema、过滤表达式、名称映射等
- DataWriteBuilder - 数据写入构建器: 扩展写入模式和指标截断长度
- DeleteWriteBuilder - 删除文件写入构建器: 位置删除和等值删除

```
// Builder ■■■■■■  
Parquet.write(outputFile)  
    .schema(icebergSchema)  
    .createWriterFunc(GenericParquetWriter::buildWriter)  
    .set("parquet.bloom.filter.column.names", "col1,col2")  
    .build();
```

设计意图: Builder 将复杂配置过程分步进行，避免构造函数参数爆炸。Parquet.java 作为 Facade 隐藏底层组件创建细节。

3.2 ParquetTypeVisitor / TypeWithSchemaVisitor - 访问者模式 (Visitor)

Schema 转换大量运用访问者模式遍历和转换树形 Schema 结构。Parquet 和 Iceberg 各有类型系统，需递归遍历进行逐类型转换。

- ParquetTypeVisitor<T> - 单 Schema 访问者，遍历 GroupType/PrimitiveType 树
- TypeWithSchemaVisitor<T> - 双 Schema 同步访问者，通过 ID 匹配遍历
- ParquetVariantVisitor<T> - Variant 专用访问者，处理分片结构

设计意图:

将遍历逻辑与具体操作解耦。相同遍历框架可服务于类型转换、读取器构建、写入器构建、ID 检查等不同操作。TypeWithSchemaVisitor 通过 ID 匹配实现双 Schema 同步遍历，特别适合 Schema 演化场景。

3.3 BaseParquetReaders / BaseParquetWriter - 模板方法模式 (Template Method)

数据层使用模板方法模式定义读写器构建骨架算法，将类型特定创建逻辑延迟到子类。

- BaseParquetReaders.createStructReader() - 创建 Struct 读取器，子类决定 Record 类型
- BaseParquetWriter.createStructWriter() - 创建 Struct 写入器

• LogicalTypeReadBuilder/WriterVisitor - 按逻辑类型分派创建具体读写器

设计意图: 不同引擎 (Spark/Flink/Trino) 使用不同内部数据格式, 模板方法允许各引擎只实现少量工厂方法即可复用整个构建流程。GenericParquetReaders/Writer 是使用 Iceberg 原生 Record 的默认实现。

3.4 三级行组过滤 - 策略模式 (Strategy)

ReadConf 初始化时依次应用三种行组过滤策略, 形成递进式过滤管线, 各策略独立可配。

过滤器	数据源	支持谓词	精度
ParquetMetricsRowGroupFilter	行组统计(min/max/count)	所有比较/IN/NOT_IN/startsWith/NaN	粗粒度
ParquetDictionaryRowGroupFilter	字典页数据	所有比较/IN/NOT_IN/startsWith	中粒度
ParquetBloomRowGroupFilter	布隆过滤器	eq/IN	细粒度

设计意图: 按从粗到细顺序执行, 统计过滤最快但不精确, 字典更精确, 布隆最精确但仅支持等值。分层设计最大化可跳过行组数。

3.5 ParquetIO - 适配器模式 (Adapter)

Iceberg 和 Parquet 各有 IO 接口, ParquetIO 作为适配器桥接两套接口。

- ParquetInputFile - 适配 Iceberg InputFile 到 Parquet InputFile
 - ParquetOutputFile - 适配 Iceberg OutputFile 到 Parquet OutputFile
 - ParquetRangeReadableInputStreamAdapter - 适配向量化 IO (readRanges)
- 设计意图: 通过适配器, Iceberg 的存储层 (HDFS/S3/GCS/Azure) 无需修改即可被 Parquet 库使用。对向量化 IO 支持允许单次系统调用完成多段数据读取。

3.6 ParquetValueReaders - 组合模式 (Composite)

值读取器使用组合模式处理嵌套数据。所有读取器实现 ParquetValueReader 接口, 复合类型持有子读取器列表。

- StructReader - 组合多字段读取器按位置分派
 - ListReader - 组合元素读取器处理 repeated group
 - MapReader - 组合键值读取器处理 key-value 重复组
 - OptionReader - 装饰器处理可空字段的 definition level
- 设计意图: 任意深度嵌套类型 (如 List<Map<String,Struct>>) 都可通过递归组合简单读取器处理。

第4章 核心流程图

4.1 Parquet 数据写入流程

用户通过 `Parquet.write()/writeData()` 创建 `Builder`，配置后 `build()` 生成 `ParquetWriter`。写入时按行组缓冲，达到目标大小后刷新。

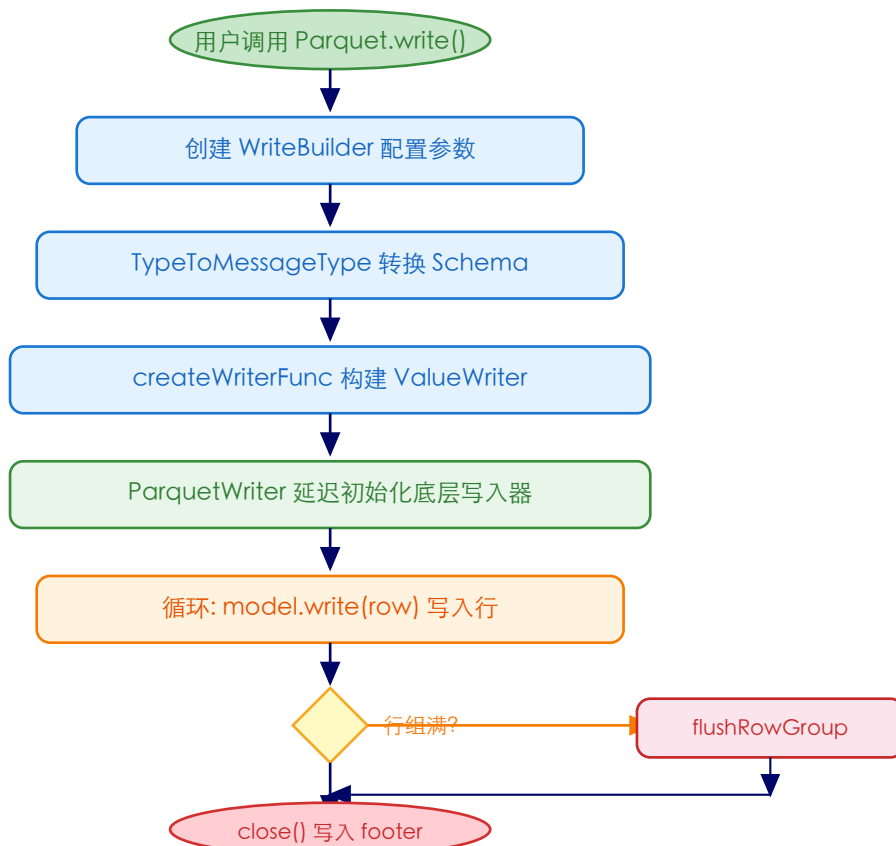


图 4-1 Parquet 数据写入流程图

4.2 Parquet 数据读取流程

读取流程从 `Parquet.read()` 开始，经历 `ReadConf` 加载、三级行组过滤、Schema 投影匹配，创建 `ParquetReader` 迭代数据。

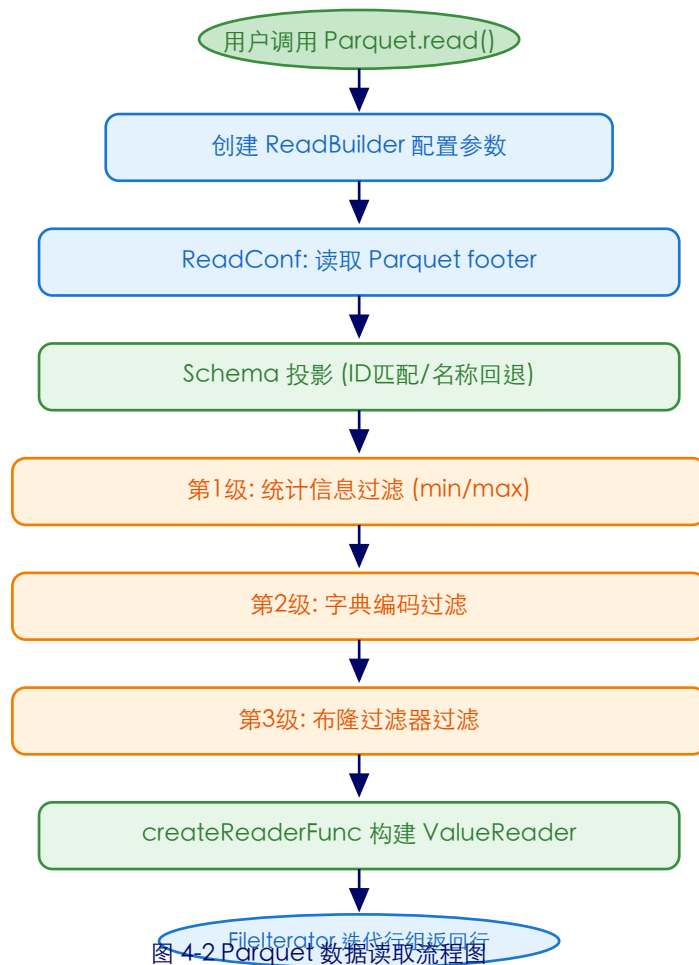


图 4-2 Parquet 数据读取流程图

4.3 三级行组过滤管线

过滤管线是读取性能优化的关键。每级过滤器独立判断行组是否可跳过，形成漏斗效应逐步缩小需读取的行组集合。

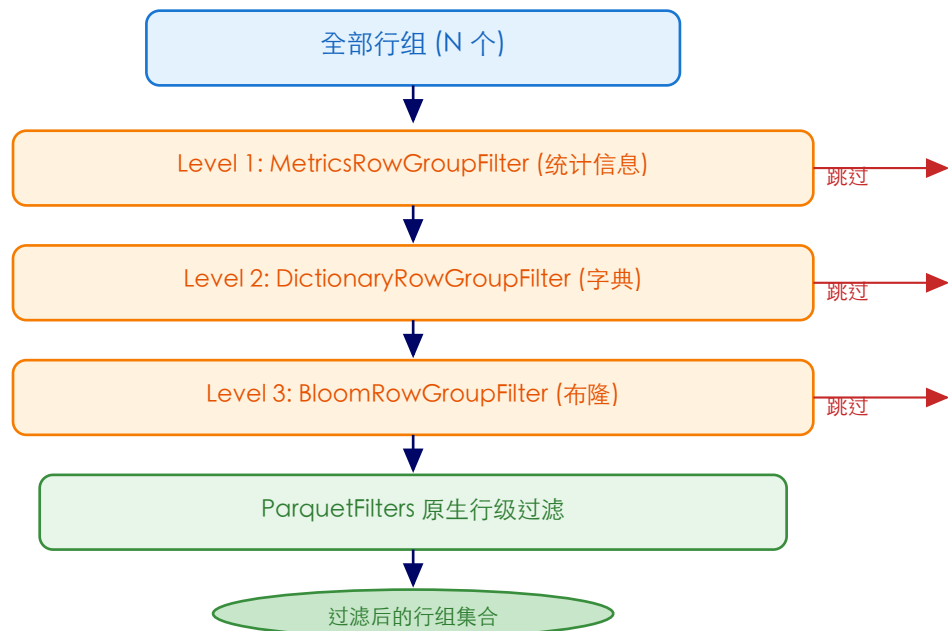


图 4-3 三级行组过滤管线

第5章 核心时序图

5.1 Parquet 文件写入时序图

展示从用户调用到数据落盘的完整交互过程。

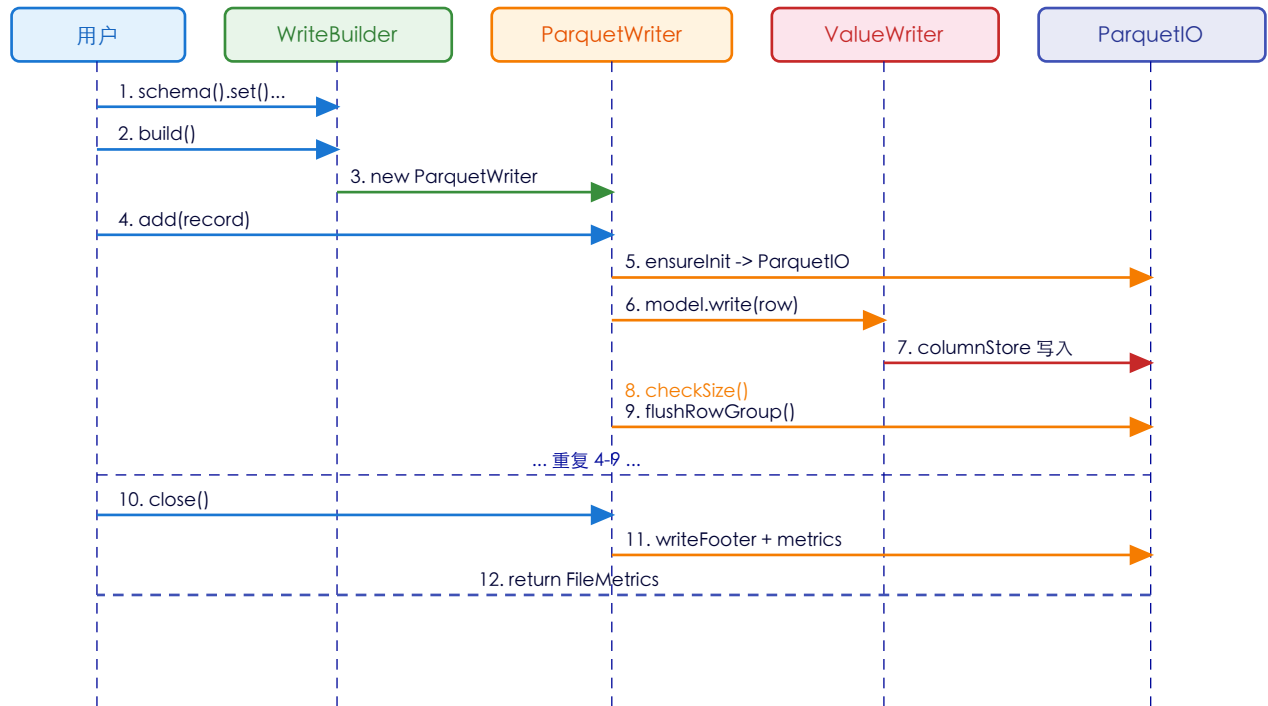


图 5-1 Parquet 文件写入时序图

5.2 Parquet 文件读取时序图

展示从用户调用到数据返回的完整交互，涉及
ReadBuilder、ReadConf、三级过滤器、ParquetReader 等。

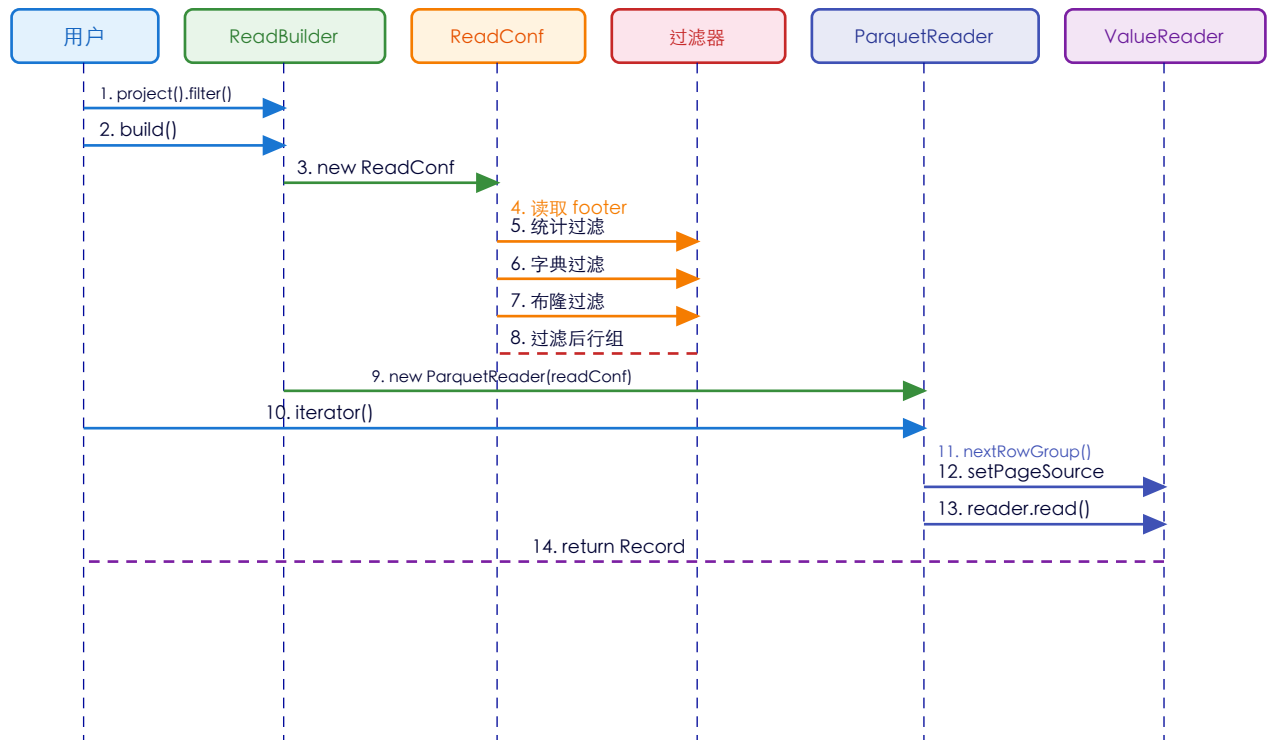


图 5-2 Parquet 文件读取时序图

5.3 Schema 转换与读取器构建时序图

展示 TypeWithSchemaVisitor 同步遍历两棵类型树，在每个节点调用 visit 方法构建 ValueReader。

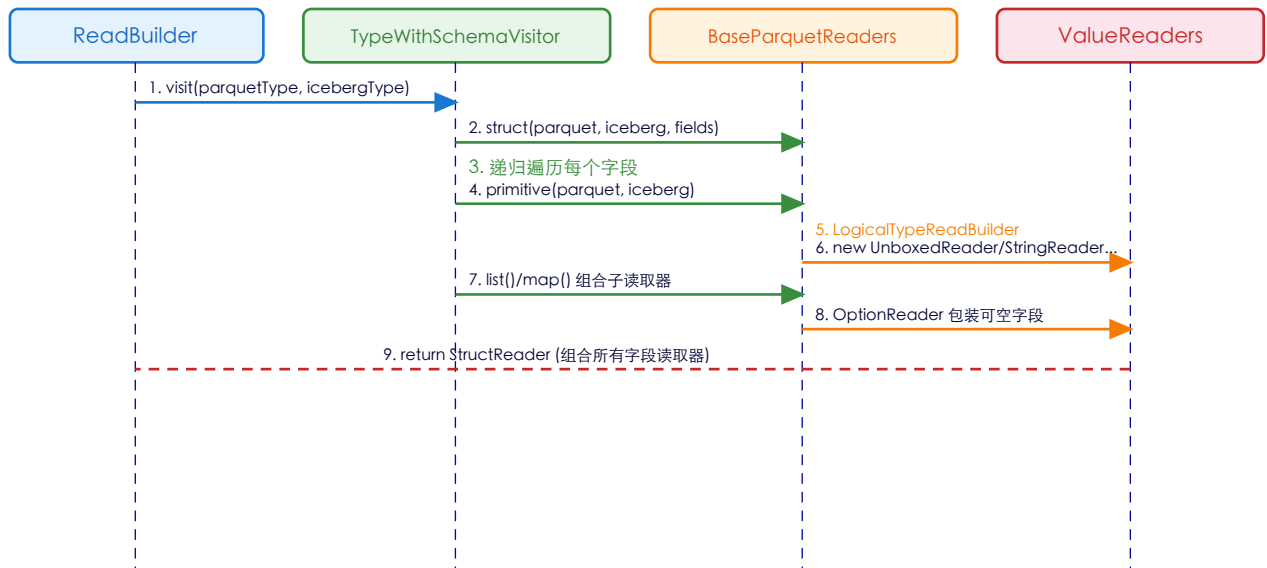


图 5-3 Schema 转换与值读取器构建时序图

第6章 关键场景调用链

6.1 场景一: 写入 Parquet 数据文件

触发条件: 用户通过 Parquet.writeData() 创建数据写入器并写入记录

步骤	类.方法	说明
1	Parquet.writeData(OutputStream)	静态工厂创建 DataWriteBuilder
2	DataWriteBuilder.schema(Schema)	设置 Iceberg Schema
3	DataWriteBuilder.createWriterFunc(fn)	注册值写入器工厂函数
4	DataWriteBuilder.build()	调用内部 WriteBuilder.build()
5	TypeToMessageType.convert(Schema)	Iceberg Schema 转 Parquet MessageType
6	ParquetWriter.add(T)	添加记录到当前行组
7	ParquetWriter.ensureWriterInitialized()	延迟初始化底层 ParquetFileWriter
8	model.write(ValueWriter, T)	通过 ValueWriter 写入列存储
9	ParquetWriter.checkSize()	检查行组大小满则 flushRowGroup
10	ParquetWriter.close()	最终刷新 + 写入 footer + 收集 metrics

6.2 场景二: 带过滤条件读取 Parquet 文件

触发条件: 用户通过 Parquet.read() 指定投影和过滤表达式读取数据

步骤	类.方法	说明
1	Parquet.read(InputStream)	创建 ReadBuilder
2	ReadBuilder.project(Schema).filter(Expression)	设置投影和过滤
3	ReadBuilder.build()	创建 ParquetReader
4	ReadConf(InputStream, ...)	读取 footer 获取行组元数据
5	ParquetSchemaUtil.pruneColumns()	裁剪文件 Schema 为投影 Schema
6	MetricsRowGroupFilter.shouldRead()	第1级: 统计信息过滤
7	DictionaryRowGroupFilter.shouldRead()	第2级: 字典编码过滤

步骤	类.方法	说明
8	BloomRowGroupFilter.shouldRead()	第3级: 布隆过滤器过滤
9	ParquetReader.FileIterator.nextRowGroup()	切换到下一个未过滤行组
10	createReaderFunc.apply(schema)	构建 ValueReader 树
11	ParquetValueReader.read(reuse)	读取当前行返回记录

6.3 场景三: Schema 演化场景读取

触发条件: 文件 Schema 与当前表 Schema 不一致（列增删改名），需 Schema 投影匹配

步骤	类.方法	说明
1	ReadConf()	读取文件 footer 获取文件 Schema
2	ParquetSchemaUtil.hasIds(fileSchema)	检查是否含 field ID
3a	pruneColumns(fileSchema, expected)	有 ID: 按 field ID 裁剪匹配
3b	pruneColumnsFallback(fileSchema, expected)	无 ID: 按名称回退匹配
4	applyNameMapping(fileSchema, nameMapping)	应用名称映射补充缺失 ID
5	TypeWithSchemaVisitor.visit()	同步遍历两棵类型树按 ID 匹配
6	BaseParquetReaders.ReadBuilder.struct()	为匹配字段创建读取器
7	NullReader/ConstantReader	为新增列创建空值/常量读取器

6.4 场景四: 写入后指标收集

触发条件: ParquetWriter.close() 后收集列级统计信息用于查询优化

步骤	类.方法	说明
1	ParquetWriter.close()	关闭写入器获取 metadata
2	ParquetMetrics.fromMetadata()	从 footer 提取全局指标
3	ParquetMetrics.fromWriterMetrics()	从字段级写入器收集指标
4	MetricsVisitor.visit(schema)	遍历 Schema 访问每列统计
5	收集 valueCounts/nullCounts/nanCounts	行数、空值数、NaN 数
6	收集 lowerBounds/upperBounds	列级 min/max 边界含截断

6.5 场景五: 向量化批量读取

触发条件: 计算引擎使用列式批量读取提高吞吐量

步骤	类.方法	说明
1	Parquet.read().createBatchedReaderFunc(fn)	注册批量读取器工厂
2	ReadBuilder.build()	创建 VectorizedParquetReader
3	VectorizedReader.setBatchSize(size)	设置每批行数
4	VectorizedReader.read(reuse)	一次读取 batchSize 行到列式内存
5	返回 ColumnarBatch	引擎在列式内存上执行向量化算子

6.6 场景六: Variant 半结构化数据读写

触发条件: 表中包含 Variant 类型列需进行分片 (Shredding) 读写

步骤	类.方法	说明
1	TypeToMessageType 遇到 VariantType	创建 variant 组 (metadata+value+typed_value)
2	VariantWriterBuilder.build()	ParquetVariantVisitor 构建写入器
3	ParquetVariantWriters 分片写入	半结构化数据拆分到各列
4	VariantReaderBuilder.build()	ParquetVariantVisitor 构建读取器
5	ParquetVariantReaders 组装 Variant	从分片列重建完整 Variant 值

第7章 配置与扩展

7.1 核心配置属性

配置项	默认值	说明
write.parquet.row-group-size-bytes	128MB	目标行组大小
write.parquet.page-size-bytes	1MB	目标页大小
write.parquet.page-row-limit	20000	每页最大行数
write.parquet.compression-codec	gzip	压缩编解码器 (gzip/snappy/lz4/zstd)
write.parquet.bloom-filter-enabled.column.*	false	启用布隆过滤器的列
write.parquet.bloom-filter-max-bytes	1MB	布隆过滤器最大字节数
write.metadata.metrics.max-inferred-column-defaults	100	自动推断指标最大列数
read.parquet.vectorization.enabled	false	启用向量化读取
read.parquet.vectorization.batch-size	5000	向量化批量大小

7.2 扩展点

Iceberg Parquet 模块提供多个扩展点允许不同计算引擎集成：

- createReaderFunc / createBatchedReaderFunc - 自定义值读取器工厂
- createWriterFunc - 自定义值写入器工厂
- BaseParquetReaders / BaseParquetWriter - 继承抽象基类覆写模板方法
- ParquetTypeVisitor - 实现自定义访问者用于特殊 Schema 遍历
- ParquetIO - 通过 Iceberg 统一 IO 层扩展存储后端
- metricsConfig - 自定义列级统计信息收集和截断长度

第8章 设计亮点与总结

8.1 设计亮点

- 三级渐进式行组过滤: 统计信息 -> 字典编码 -> 布隆过滤器，从粗到细的漏斗效应，在各种查询场景下有效跳过不相关数据。三个过滤器各自独立可灵活启用。
- 访问者模式精妙运用: ParquetTypeVisitor 和 TypeWithSchemaVisitor 统一 Schema 遍历框架，使得类型转换、读取器构建、写入器构建等操作只需实现 visit 方法。TypeWithSchemaVisitor 的双 Schema 同步遍历特别适合演化场景。
- 延迟初始化策略: ParquetWriter 在第一条数据到达时才初始化，ParquetReader 按需加载行组，减少资源浪费，支持流式处理。
- 模板方法实现引擎解耦: BaseParquetReaders/Writer 定义构建骨架，各引擎只需实现少量工厂方法即可复用整个读写流程，体现了开闭原则。
- IO 适配层设计: ParquetIO 将存储后端与文件格式解耦，支持 HDFS/S3/GCS/Azure 等，向量化 IO 适配器允许单次系统调用完成多段读取。
- Variant 分片读写: 对半结构化数据进行 Shredding，将频繁访问的字段提升为独立列以获得列式存储的性能优势，是对 JSON 类数据的先进处理方式。

8.2 设计限制

- Parquet 库深度耦合: 底层依赖 Apache Parquet Java 库的内部 API，版本升级可能带来兼容性挑战
- 行组过滤粒度: 过滤仅在行组级别，无法跳过行组内的特定数据页
- 布隆过滤器限制: 仅支持等值查询 (eq/IN)，不支持范围查询
- Variant 功能较新: Variant 支持尚处于发展阶段，API 可能发生变化

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	四层架构清晰，组件职责单一，层间通过接口解耦
设计模式运用	★★★★★	Builder/Visitor/Template/Strategy/Adapter/Composite 运用恰当
可扩展性	★★★★★	多个扩展点允许不同引擎集成，模板方法保证复用
性能优化	★★★★★	三级过滤、向量化读取、延迟初始化、自适应行组刷新
类型完整性	★★★★☆	覆盖所有 Iceberg 类型含 Variant，边界类型处理完善
文档完整度	★★★☆☆	代码注释较少，部分复杂逻辑缺乏文档说明